

Lekce 1: Vítejte, přehled kurzu a úvod do Node.js

1. Osnova obsahu

A. Vítání a představení

- **Privítání a představení instruktora:**
 - Krátké osobní představení.
 - Přehled vašich zkušeností a odbornosti v oblasti testování a Cypressu.
- **Představení studentů:**
 - Rychlé kolečko představování (jméno, zázemí, očekávání).

B. Přehled kurzu

- **Cíle kurzu:**
 - Přejít z manuálního na automatizované testování.
 - Zvládnutí základů frameworku Cypress.
- **Struktura kurzu:**
 - Rozdělení na moduly a lekce.
 - Přehled praktických činností a závěrečného projektu.
- **Očekávání a požadavky:**
 - Docházka a aktivní účast.
 - Závazek dokončit zadané úkoly a cvičení.

C. Úvod do automatizovaného testování

- **Co je automatizované testování?**
 - Definice a srovnání s manuálním testováním.
- **Výhody automatizovaného testování:**
 - Efektivita a rychlost.
 - Znovupoužitelnost a konzistence.
 - Lepší pokrytí testů a spolehlivost.
- **Typy automatizovaného testování:**
 - Jednotkové testy (Unit Testing), integrační testy, end-to-end (E2E) testování.

Výhody automatizovaného testování

Definice:

Automatizované testování znamená použití specializovaných nástrojů a skriptů ke spouštění testů na softwarových aplikacích automaticky, bez manuálního zásahu.

Podrobné vysvětlení:

1. Efektivita a rychlost:

- **Rychlé provedení:** Automatizované testy běží podstatně rychleji než manuální, poskytují rychlou zpětnou vazbu na změny v kódu.
- **Časté testování:** Umožňují spouštění testů vícekrát denně, což je nezbytné pro CI/CD pipeline.

2. Znovupoužitelnost a konzistence:

- **Znovupoužitelné skripty:** Testovací skripty lze použít v různých projektech a scénářích, což snižuje náročnost vytváření nových testů.
- **Konzistentní výsledky:** Automatizované testy pokaždé vykonávají stejné kroky, čímž odstraňují lidské chyby a zajišťují spolehlivé výsledky.

3. Lepší pokrytí testů:

- **Komplexní testování:** Umožňuje testovat širokou škálu scénářů včetně krajních případů (edge cases), na které by se v manuálním testování mohlo zapomenout.
- **Regresní testování:** Rychlé ověření, že nové změny kódu neovlivnily stávající funkcionality.

4. Nákladová efektivita:

- **Snížené náklady na práci:** Počáteční nastavení může být náročnější, ale z dlouhodobého hlediska automatizace šetří čas i zdroje díky minimalizaci potřeby opakovaného manuálního testování.
- **Včasné odhalení chyb:** Odhalení a oprava chyb v rané fázi vývoje snižuje náklady na jejich odstranění po vydání.

5. Vyšší přesnost:

- **Preciznost:** Automatizované testy vykonávají předdefinované kroky přesně a nevynechávají žádné detaily, což zajišťuje důkladné ověření chování aplikace.
- **Spolehlivá práce s daty:** Konzistentně spravují testovací data i prostředí a poskytují důvěryhodné výsledky testů.

6. Škálovatelnost:

- **Podpora velkých projektů:** Snadno zvládají rozsáhlé sady testů a komplexní aplikace bez nutnosti úměrně navyšovat úsilí.

7. CI/CD (kontinuální integrace a doručení):

- **Bezproblémová integrace:** Testy lze jednoduše zapojit do CI/CD pipeline, umožňují průběžné testování a rychlé nasazení změn.

- **Okamžitá zpětná vazba:** Vývojáři ihned ví, zda změny kódu testy prošly.

D. Přehled Cypressu jako testovacího frameworku

- **Co je Cypress?**
 - Úvod do Cypressu a jeho role v E2E testování.
- **Proč zvolit Cypress?**
 - Klíčové funkce: okamžité načtení změn, automatické čekání, snadné ladění.
 - Srovnání s jinými nástroji (např. Selenium).

Co je Cypress?

Definice:

Cypress je open-source end-to-end testovací framework navržený speciálně pro moderní webové aplikace. Umožňuje vývojářům i QA inženýrům snadno psát, spouštět a ladit testy přímo v prohlížeči.

Podrobné vysvětlení:

1. End-to-End testovací framework:

- **Kompletní testování:** Cypress umožňuje testování celých uživatelských scénářů, od interakcí na UI až po odezvy backendové API.

2. Přátelské pro vývojáře:

- **Jednoduché nastavení:** K použití Cypressu je potřeba minimální konfigurace, což zpřístupňuje framework i začátečníkům v automatizovaném testování.
- **Okamžité načítání změn:** Cypress automaticky detekuje změny v testech a ihned je znovu spustí, což urychluje vývoj.

3. Navrženo pro moderní webové aplikace:

- **Založeno na JavaScriptu:** Cypress je psán v JavaScriptu a přirozeně se integruje s frameworky jako React, Angular, Vue.js.
- **Moderní architektura:** Narozdíl od tradičních nástrojů běží Cypress přímo v prohlížeči.

4. Silné ladicí nástroje:

- **Interaktivní běžec testů:** Poskytuje vizuální rozhraní s možností sledovat testy v reálném čase, inspektovat prvky a prohlížet podrobné logy.
- **Time Travel:** Poskytuje možnost krokovat (hover) jednotlivými příkazy a zjistit, co se stalo v každém kroku.

5. Automatické čekání:

- **Bez ručního čekání:** Cypress automaticky čeká na načtení prvků a dokončení příkazů, snižuje potřebu manuální synchronizace.

6. Kontrola síťové komunikace:

- **Zachytávání a mockování požadavků:** Cypress umožňuje zachytávat a měnit síťové požadavky, což usnadňuje testování různých scénářů a API.

7. Rozšiřitelnost:

- **Pluginy a integrace:** Podporuje širokou škálu rozšíření a snadno se integruje s dalšími nástroji dle potřeb projektu.

8. Podrobná dokumentace:

- **Návody a příklady:** Cypress nabízí obsáhlou dokumentaci a příklady použití.

Proč zvolit Cypress?

Definice:

Cypress se vyznačuje unikátní architekturou, snadným používáním a robustní sadou funkcí přizpůsobenou modernímu vývoji webu.

1. Rychlost a efektivita:

- **Rychlé provádění testů:** Spouští testy přímo v prohlížeči s rychlou odezvou a snižuje celkový čas testování.
- **Okamžité načtení změn:** Testy se po úpravě automaticky načítají, zvyšuje produktivitu.

2. Jednoduché nastavení a používání:

- **Snadná instalace:** Cypress lze nainstalovat během několika minut přes npm.
- **Minimální konfigurace:** K zahájení psaní testů stačí jen základní nastavení.

3. Okamžitá zpětná vazba a ladění:

- **Interaktivní test runner:** Zobrazí testy v reálném čase, s možností interakce a inspekce.
- **Time Travel ladění:** Možnost krokovat příkazy pro snadnější identifikaci chyb.

4. Automatické čekání:

- **Vestavěná synchronizace:** Cypress čeká na načtení prvků a dokončí příkazy než pokračuje, což zjednodušuje testy a eliminuje "flaky" chování.

5. Síťová kontrola a mockování:

- **Zachytávání požadavků:** Umožňuje manipulaci se síťovými požadavky pro simulaci scénářů a mockování odpovědí backendu.

6. Bohatá sada asercí a příkazů:

- **Vestavěné aserce:** Usnadňuje ověřování chování aplikací.
- **Vlastní příkazy:** Lze definovat vlastní příkazy pro opakované úkony.

7. Integrace s CI/CD:

- **Kompatibilita:** Hladce spolupracuje s CI systémy jako GitHub Actions, GitLab CI, Jenkins.
- **Cypress Dashboard:** Pokročilá analytika a monitoring testů.

8. Silná komunita a podpora:

- **Aktivní komunita:** Přispívá pluginy i radami.
- **Obsáhlá dokumentace:** Uživatelé mají k dispozici podrobné návody a řešení.

9. Moderní architektura:

- **Přímá interakce s prohlížečem:** Přináší přesnější a spolehlivější výsledky než nástroje mimo prohlížeč.
- **Kompatibilita s moderními frameworky:** Ideální pro dnešní dynamické webové aplikace.

10. Udržovatelnost a škálovatelnost:

- **Čitelný testovací kód:** Podporuje psaní přehledných a snadno udržovatelných testů.
- **Znovupoužitelné komponenty:** Možnost tvorby opakovatelných částí testů.

E. Úvod do Node.js

- **Co je Node.js?**
 - Přehled Node.js a jeho důležitosti v moderním webovém vývoji.
- **Node.js a Cypress:**
 - Jak Cypress využívá Node.js ke spouštění testů.

Co je Node.js?

Definice:

Node.js je open-source multiplatformní JavaScript runtime, který umožňuje vývojářům spouštět JavaScript mimo webový prohlížeč, typicky na serveru.

Podrobné vysvětlení:

1. JavaScriptové běhové prostředí:

- **Serverový JavaScript:** Spouští JavaScript na serveru, což umožňuje stavět škálovatelné síťové aplikace.
- **V8 Engine:** Stojí na Chrome V8 enginu, poskytuje vysoký výkon.

2. Událostmi řízený neblokující I/O:

- **Asynchronní operace:** Událostně řízený model s neblokujícím I/O efektivně zvládá mnoho současných požadavků.
- **Škálovatelnost:** Toto řešení umožňuje zpracovat velké množství požadavků bez velké zátěže.

3. Správa balíčků pomocí npm:

- **Široký ekosystém:** Node.js je dodáván s npm – největším ekosystémem open-source knihoven a nástrojů.
- **Správa závislostí:** Zjednodušuje správu závislostí projektů.

4. Multiplatformní kompatibilita:

- **Podpora více OS:** Funguje na Windows, macOS i Linuxu.
- **Jednotný jazyk:** Umožňuje vývoj jak na klientu, tak serveru v JavaScriptu.

5. Široké využití:

- **Web servery a API:** Vhodný pro rychlé a škálovatelné web servery, REST API i reálné aplikace jako chat nebo online hry.
- **Příkazové nástroje:** Umožňuje tvorbu CLI utilit a skriptů.
- **Mikroservisy:** Umožňuje tvorbu udržitelných, modulárních aplikací.

6. Aktivní komunita a vývoj:

- **Živý ekosystém:** Aktivní komunita neustále zlepšuje Node.js.
- **Pravidelné aktualizace:** Zajišťují bezpečnost a moderní přístup.

7. Integrace s moderními nástroji:

- **Build nástroje:** Webpack, Gulp, Grunt pro efektivní buildy a automatizaci.
- **Testovací frameworky:** Podpora nástrojů jako Mocha, Jest, Cypress.

8. Výkon a efektivita:

- **Vysoká propustnost:** Optimalizován pro I/O-intenzivní operace.
- **Nízká latence:** Minimalizuje odezvy pro uživatele.

Jak Cypress využívá Node.js ke spouštění testů

Definice:

Cypress používá Node.js jako své runtime prostředí pro spouštění testovacích skriptů, správu závislostí a integraci s dalšími nástroji, což zvyšuje výkon i možnosti frameworku.

Podrobné vysvětlení:

1. Provozní prostředí:

- **Spouštění testovacích skriptů:** Testy napsané v JavaScriptu se provádějí v Node.js.
- **Serverové úkony:** Node.js zajišťuje např. obsluhu síťových požadavků během testování.

2. Správa balíčků pomocí npm:

- **Správa závislostí:** Cypress se instaluje přes npm, umožňuje aktualizace i rozšíření.
- **Automatizace skriptů:** Pomocí npm skriptů lze automatizovat spouštění testů.

3. Integrace s vývojářskými nástroji:

- **Build nástroje a task runnery:** Snadná integrace s Webpackem a Gulpem.
- **CI/CD pipelines:** Node.js umožňuje hladké zapojení Cypressu do Continuous Integration.

4. Systém pluginů:

- **Rozšiřování funkcí:** Cypress pluginy (Node.js moduly) nabízí rozšíření podle potřeb projektu.
- **Přínos komunity:** Široká paleta pluginů z komunity (kódové pokrytí, vizuální testování...)

5. Práce se souborovým systémem:

- **Čtení/zápis souborů:** Node.js umožňuje práci s testovacími daty, logy apod.
- **Konfigurace prostředí:** Načítání a aplikování konfigurací, např. z `cypress.config.js`.

6. Síťová komunikace a testování API:

- **Zachytávání požadavků:** Umožňuje mockování a manipulaci s API v průběhu testů.
- **Backend interakce:** Možnost komplexního testování frontend i backend částí.

7. Optimalizace výkonu:

- **Efektivní spouštění testů:** Neblokující architektura Node.js zajišťuje efektivní provádění i při více požadavcích.
- **Správa zdrojů:** Efektivní využití HW prostředků při testování.

8. Režim živých aktualizací (Watch mode):

- **Okamžité aktualizace:** Node.js zajišťuje live reload při změně kódu nebo testů.
- **Interaktivní vývoj:** Umožňuje ihned sledovat efekt změn v testech.

9. Multiplatformní podpora:

- **Jednotnost napříč OS:** Cypress běží stejně na Windows, macOS i Linuxu.
- **Flexibilita nasazení:** Možnost provozu lokálně i v cloudových CI systémech.

F. Nastavení vývojového prostředí

- **Instalace potřebných nástrojů:**
 - **Node.js a npm:**
 - Krok za krokem průvodce instalací.
 - Ověření instalace v příkazovém řádku.
 - **Kódový editor (VS Code):**
 - Úvod do Visual Studio Code.
 - Důležité rozšíření pro Cypress a JavaScript.
 - **Git:**
 - Instalace Gitu.
 - Základní příkazy a nastavení repozitáře na GitHubu/GitLabu.

G. Začínáme s GitHub/GitLab

- **Vytvoření repozitáře:**
 - Průvodce vytvořením nového repozitáře.
- **Klonování repozitáře:**
 - Klonování na lokální počítač.
- **Základní operace s Gitem:**
 - `git add` , `git commit` , `git push` .

H. Přehled průběhu lekce

- **Agenda lekce:**
 - Co očekávat v příštích třech hodinách.
- **Otázky a odpovědi:**

- Prostor pro dotazy.

2. Praktické aktivity: Cvičení a webové funkce

A. Instalace a ověření Node.js a npm

- **Cvičení:**
 - Studenti si nainstalují Node.js z nodejs.org.
 - Ověření instalace pomocí:
 - `node -v`
 - `npm -v`
- **Webová funkce:**
 - Vytvořit jednoduchý JavaScript soubor (`hello.js`), který vypíše "Hello, Cypress!" do konzole.
 - Skript spustit pomocí `node hello.js` .

B. Nastavení Gitu a repozitáře na GitHub/GitLab

- **Cvičení:**
 - Instalace Gitu z git-scm.com.
 - Nastavit Git pomocí `git config --global user.name "Vaše jméno"` a `git config --global user.email "vy@prikklad.cz"` .
 - Vytvořit nový repozitář na GitHubu nebo GitLabu.
 - Klonovat repozitář na lokální stroj pomocí `git clone <adresa-repozitáře>` .
- **Webová funkce:**
 - Vytvořit `README.md` s krátkým popisem projektu.
 - Přidat, commitnout a pushnout `README.md` na GitHub/GitLab.

C. Úvod do Visual Studio Code (VS Code)

- **Cvičení:**
 - Instalace VS Code z code.visualstudio.com.
 - Instalace důležitých rozšíření:
 - **Cypress Snippets** pro příkazy Cypressu.
 - **ESLint** pro kontrolu JavaScriptu.
- **Webová funkce:**
 - Otevřít klonovaný repozitář ve VS Code.
 - Vytvořit základní HTML soubor (`index.html`).
 - Přidat JavaScript pro manipulaci s DOM (např. úprava textu na stránce).

D. Základní práce s Gitem

- **Cvičení:**
 - Provést změnu v `README.md` (např. přidat sekci "Začínáme").
 - Změnu přidat do stage pomocí `git add README.md`.
 - Commitnout změnu příkazem `git commit -m "Přidána sekce Začínáme"`.
 - Pushnout změnu na GitHub/GitLab pomocí `git push`.
- **Webová funkce:**
 - Zkontrolovat změny v online repozitáři.

3. Možné otázky studentů

1. Obecné dotazy k kurzu:

- Jaké předchozí znalosti potřebuji pro tento kurz?
- Jak mi kurz pomůže v současné práci?
- Jak nejlépe sledovat a studovat materiály kurzu?

2. Automatizované testování:

- Jaký je praktický rozdíl mezi manuálním a automatizovaným testováním?
- Může automatizace plně nahradit manuální testování?

3. Framework Cypress:

- Proč zvolit Cypress místo Selenium?
- Jaká jsou omezení Cypressu?

4. Node.js:

- Proč je Node.js nezbytný pro Cypress?
- Musím detailně umět Node.js pro efektivní práci s Cypress?

5. Nastavení vývojového prostředí:

- Co když narazím na chyby při instalaci Node.js nebo npm?
- Existují alternativy k VS Code?

6. Git a GitHub/GitLab:

- Jak důležitá je správa verzí v testovací automatizaci?
- Co dělat, pokud omylem smažu svůj lokální repozitář?

7. Praktická cvičení:

- Jak mohu mimo cvičení trénovat dále?
- Budeme pracovat na reálných projektech v průběhu kurzu?

8. Logistika kurzu:

- Kolik času týdně mám kurzu věnovat?
- Budou k dispozici záznamy a práce na domácích úkolech?

4. Doporučené doplňující materiály

A. Oficiální dokumentace a návody:

- Cypress dokumentace:
 - [Cypress průvodce začátkem](#)
- Node.js dokumentace:
 - [Oficiální dokumentace Node.js](#)
- Git dokumentace:
 - [Oficiální dokumentace Git](#)

B. Tutoriály a články:

- Základy JavaScriptu:
 - [MDN JavaScript Guide](#)
- Git a GitHub tutoriál:
 - [GitHub Learning Lab](#)
- VS Code tutoriály:
 - [Dokumentace VS Code](#)

C. Interaktivní výukové platformy:

- Codecademy:
 - [Kurzy JavaScriptu](#)
- FreeCodeCamp:
 - [JavaScript algoritmy a datové struktury](#)
- GitHub Learning Lab:
 - [Úvod do GitHubu](#)

D. Video a webináře:

- Cypress YouTube kanál:
 - [Cypress.io na YouTube](#)
- Node.js tutoriály:
 - [Traversy Media – Node.js Crash Course](#)
- Git tutoriály:
 - [Academind – Git Tutorial for Beginners](#)

E. Komunita a podpora:

- Cypress komunita:

- [Cypress diskusní fórum](#)
- **Stack Overflow:**
 - [Tag Cypress na Stack Overflow](#)
- **Komunity GitHub/GitLab:**
 - Zapojte se do diskuzí a řešení problémů v repozitářích – praktické učení.

5. Navržený rozpis lekce na 3 hodiny

1. hodina: Úvod a přehled kurzu (60 minut)

- **Úvod (15 minut):**
 - Představení instruktora a studentů.
- **Přehled kurzu (20 minut):**
 - Cíle, struktura, očekávání.
- **Úvod do automatizovaného testování (25 minut):**
 - Co je automatizace?
 - Výhody a typy testů.

2. hodina: Úvod do Cypressu a Node.js (60 minut)

- **Přehled Cypressu (30 minut):**
 - Co je Cypress?
 - Klíčové vlastnosti a srovnání frameworků.
- **Úvod do Node.js (30 minut):**
 - Co je Node.js?
 - Role v Cypressu.
 - Krátká ukázka Node.js v praxi.

3. hodina: Nastavení prostředí a základy GitHub/GitLab (60 minut)

- **Nastavení nástrojů (30 minut):**
 - Instalace Node.js a npm.
 - Instalace VS Code a rozšíření.
 - Instalace Git.
- **Základy GitHub/GitLab (20 minut):**
 - Vytvoření a klonování repo.
 - Základní příkazy Gitu.
- **Q&A sekce (10 minut):**

- Prostor pro okamžité dotazy.

6. Dodatečná doporučení

- **Interaktivní demonstrace:**
 - Živá ukázka instalace Node.js a jeho ověření.
 - Průchod založením repo a základními git operacemi.
- **Poutavá vizuální podpora:**
 - Využijte diagramy k vysvětlení vztahu Node.js, Cypress a testovací workflow.
 - Snímky obrazovky kroků instalace.
- **Podpora zapojení:**
 - Vyzvěte studenty, ať sdílí obrazovku během cvičení.
 - Podporujte otevřenou a kooperativní atmosféru.
- **Jasně pokyny:**
 - Všechny kroky popsat jasně a jednoduše.
 - Přidat tipy na řešení obvyklých problémů při instalaci.

Odpovědi pro klíčové sekce

Část o automatizaci testování:

Otázka: Co je automatizované testování a jak se liší od manuálního?

Odpověď:

Automatizované testování využívá specializované nástroje a skripty pro automatické provádění testů softwaru, bez zásahu člověka. Na rozdíl od manuálního testování, kdy tester sám krok po kroku provádí testovací případy, automatizace spouští předem připravené skripty, které opakovaně a konzistentně provádějí stejné akce. Tento přístup je efektivnější, méně náchylný k chybám a poskytuje rychlejší zpětnou vazbu na změny v kódu, což je ideální pro časté cykly testování v agilním vývoji.

Otázka: Jaké jsou hlavní přínosy zařazení automatizace testování do projektu?

Odpověď:

Automatizace testování přináší řadu výhod včetně:

- **Vyšší efektivita:** Testy běží rychleji a lze je spouštět opakovaně s minimálním úsilím.

- **Lepší přesnosti:** Odstraňuje lidské chyby, zaručuje konzistentní a spolehlivé výsledky.
- **Úspory nákladů:** Snižuje dlouhodobé náklady na ruční testování a opravy chyb.
- **Komplexní pokrytí testů:** Umožňuje testování řady scénářů včetně okrajových případů.
- **Podpora CI/CD:** Lze snadno integrovat do pipeline, testují se automaticky při každé změně kódu.

Část o frameworku Cypress:

Otázka: Co je Cypress a čím se liší od jiných frameworků jako Selenium?

Odpověď:

Cypress je open-source end-to-end testovací framework pro moderní webové aplikace. Na rozdíl od Selenium, které běží mimo prohlížeč a komunikuje s ním přes ovladače, Cypress běží přímo uvnitř prohlížeče. Díky tomu je výrazně rychlejší, spolehlivější a nabízí lepší možnosti ladění. Cypress dále poskytuje okamžité načítání změn, automatické čekání a interaktivní běžec testů s vizuálním rozhraním, které je uživatelsky mnohem přívětivější.

Otázka: Jaké klíčové vlastnosti má Cypress, které zlepšují zážitek z testování?

Odpověď:

Mezi klíčové vlastnosti Cypressu patří:

- **Okamžité načítání změn:** Po změně testu se automaticky spustí.
- **Automatické čekání:** Není potřeba ručně zadávat čekání na načtení prvků.
- **Interaktivní běžec testů:** Zobrazuje testy v reálném čase, s možností inspekce a ladění.
- **Time Travel ladění:** Možnost zpětně procházet jednotlivé příkazy v průběhu testu.
- **Kontrola síťové komunikace:** Možnost zachytávat a mockovat síťové požadavky.

Část o Node.js:

Otázka: Co je Node.js a proč je důležitý pro moderní webový vývoj?

Odpověď:

Node.js je open-source multiplatformní JavaScriptové běhové prostředí umožňující spouštět JavaScript mimo prohlížeč, typicky na serveru. Je postaven na engine Chrome V8 a umožňuje škálovatelný vývoj síťových aplikací díky svému event-driven, neblokujícímu I/O modelu. Je vhodný pro tvorbu rychlých web serverů, reálných aplikací a práce s mnoha současnými připojeními s minimální režií. Jeho bohatý ekosystém (npm) přináší spoustu knihoven, které zrychlují vývoj.

Otázka: Jak využívá Cypress Node.js k efektivnímu spouštění testů?

Odpověď:

Cypress využívá Node.js jako běhové prostředí pro spouštění test skriptů, správu závislostí a integraci s dalšími nástroji. Node.js zajišťuje serverové operace potřebné pro Cypress (např. obsluha síťových požadavků), stejně jako instalaci a správu Cypressu přes npm. Díky event-driven architektuře může Cypress efektivně provádět mnoho operací současně. Node.js také umožňuje jednoduchou integraci Cypressu do CI/CD pipeline a automatizované testování v různých prostředích.